

ASSESSMENT TOOL - III : CODING CHALLENGE

Course Code	ECA1422
Course Name	Embedded Systems
Course Outcome	CO2 -
Assessment Tool	Coding Challenge
Question 1 No.	
Question 1 Title	Write an Embedded C application demonstration bit manipulation for GPIO Control and analyze efficiency. <i>oil</i>
Question 2 No.	
Question 2 Title	Design a real-time Embedded C application scheduling multiple tasks using timer interrupts.
Maximum Marks	100 (50 + 50)
Bloom's Taxonomy Level	BL3 / BL4

Evaluation Rubric - Question 1 (50 Marks)

Criteria	Weightage	Level 4	Level 3	Level 2	Level 1	Marks Awarded
Problem Understanding	10	☒ 10	☐ 8	☐ 6	☐ 4	10
Algorithm	12.5	☒ 12.5	☐ 10	☐ 8	☐ 5	12.5
Code Implementation	10	☒ 10	☐ 8	☐ 6	☐ 4	10
Competitive Performance	10	☒ 10	☐ 8	☐ 6	☐ 4	10
Output Validation	7.5	☒ 7.5	☐ 6	☐ 4	☐ 2	7.5
Total - Question 1	50					50 / 50

Evaluation Rubric - Question 2 (50 Marks)

Criteria	Weightage	Level 4	Level 3	Level 2	Level 1	Marks Awarded
Problem Understanding	10	☒ 10	☐ 8	☐ 6	☐ 4	10
Algorithm	12.5	☒ 12.5	☐ 10	☐ 8	☐ 5	12.5
Code Implementation	10	☒ 10	☐ 8	☐ 6	☐ 4	10
Competitive Performance	10	☒ 10	☐ 8	☐ 6	☐ 4	10
Output Validation	7.5	☒ 7.5	☐ 6	☐ 4	☐ 2	7.5
Total - Question 2	50					50 / 50

Code of Conduct:

I M. Jahnvi / 192411196 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Jahnvi

Embedded C Programming Assignment

GPIO Bit Manipulation and Real-Time Task Scheduling

Question 1: GPIO Bit Manipulation (BL3, 50 Marks)

Write an Embedded C program demonstrating bit manipulation for GPIO control and analyze its efficiency.

1.1 Objective

General Purpose Input/Output (GPIO) pins in microcontrollers are controlled by writing directly to memory-mapped registers. Each bit of a GPIO register corresponds to one physical pin, so setting, clearing, and reading a pin's state is done using bitwise operators instead of arithmetic operators. This program simulates a single-byte GPIO port register (GPIO_PORT) and implements three standard bit-manipulation routines used in real embedded systems: GPIO_SET, GPIO_CLEAR, and GPIO_READ.

1.2 Source Code

```
#include <stdio.h>
#include <stdint.h>

// Simulated GPIO register
uint8_t GPIO_PORT = 0x00;

// GPIO pin number
#define LED_PIN 2

// Set a GPIO bit
void GPIO_SET(uint8_t pin)
{
    GPIO_PORT |= (1 << pin);
}

// Clear a GPIO bit
void GPIO_CLEAR(uint8_t pin)
{
    GPIO_PORT &= ~(1 << pin);
}

// Read a GPIO bit
int GPIO_READ(uint8_t pin)
{
    return (GPIO_PORT >> pin) & 1;
}

int main()
{
    printf("=== GPIO BIT MANIPULATION ===\n\n");
    printf("Initial GPIO Register: 0x%02X\n", GPIO_PORT);
```

```
// Set GPIO 2
GPIO_SET(LED_PIN);
printf("\nSetting GPIO 2...\n");
printf("GPIO Register: 0x%02X\n", GPIO_PORT);
printf("LED State: ON\n");

// Read GPIO 2
printf("GPIO 2 Value: %d\n", GPIO_READ(LED_PIN));

// Clear GPIO 2
GPIO_CLEAR(LED_PIN);
printf("\nClearing GPIO 2...\n");
printf("GPIO Register: 0x%02X\n", GPIO_PORT);
printf("LED State: OFF\n");

// Read GPIO 2
printf("GPIO 2 Value: %d\n", GPIO_READ(LED_PIN));

return 0;
}
```

1.3 Program Output

The program was compiled and executed successfully on OnlineGDB (C compiler). The captured console output is shown below.

```
=== GPIO BIT MANIPULATION ===

Initial GPIO Register: 0x00

Setting GPIO 2...
GPIO Register: 0x04
LED State: ON
GPIO 2 Value: 1

Clearing GPIO 2...
GPIO Register: 0x00
LED State: OFF
GPIO 2 Value: 0

...Program finished with exit code 0
```

1.4 Explanation of Bit Manipulation Logic

- `GPIO_SET(pin)`: Uses the bitwise OR-assignment operator with a left-shifted mask, `GPIO_PORT |= (1 << pin)`. The shift produces a mask with a single 1 at the target bit position; ORing it into the register sets that bit to 1 while leaving every other bit unchanged.

- `GPIO_CLEAR(pin)`: Uses `GPIO_PORT &= ~(1 << pin)`. The mask is bitwise-inverted, producing all 1s except a 0 at the target position; ANDing it with the register forces that single bit to 0 without disturbing the rest of the register.
- `GPIO_READ(pin)`: Uses `(GPIO_PORT >> pin) & 1`. Right-shifting moves the target bit into the least-significant position, and masking with 1 isolates it, returning either 0 or 1.

Because each routine touches only the bit it is meant to touch, several GPIO pins on the same port can be managed independently and safely by different parts of a program — a requirement in real microcontrollers where one 8/16/32-bit register controls many physical pins simultaneously.

1.5 Efficiency Analysis

Time complexity: Every operation (SET, CLEAR, READ) executes in constant time, $O(1)$. Each routine consists of a fixed, small number of machine-level instructions — a shift, a logical operation, and a store or return — regardless of register width or which pin is addressed.

Space complexity: $O(1)$. Only the single `GPIO_PORT` variable and the `pin` argument are used; no arrays, loops, or dynamic memory are required.

Why bitwise operators are efficient for GPIO control:

1. **Direct hardware mapping:** Bitwise operations translate almost one-to-one into single machine instructions (AND, OR, NOT, SHIFT) on real hardware, which is why compilers generate very compact and fast code for them.
2. **Atomicity of intent:** `|=`, `&=`, and `>> &` operate on the whole register in one instruction, avoiding the need to read, modify, and check individual bits through slower arithmetic (e.g. division/modulo) or loops.
3. **Non-destructive updates:** Because the mask only affects the target bit, other pins/flags packed into the same register are left untouched — critical for GPIO ports where 8 pins commonly share one register.
4. **Predictable execution time:** Unlike branch-heavy or loop-based approaches, bitwise operations have fixed, deterministic timing, which matters in real-time and interrupt-driven embedded code where jitter must be minimized.
5. **Low memory and power footprint:** No additional buffers or lookup tables are needed, which is important on resource-constrained microcontrollers with limited RAM and flash.

Overall, the bitwise approach used here is the standard, industry-recommended technique for GPIO control in embedded C because it offers $O(1)$ time and space complexity, precise single-pin control, and instruction-level efficiency that maps directly onto microcontroller hardware registers.

Question 2: Real-Time Task Scheduler Using Timer Interrupts (50 Marks)

Design a real-time Embedded C application scheduling multiple tasks using timer interrupts.

2.1 Objective

Real-time embedded systems frequently need to run several tasks at different periodic rates using a single hardware timer. This is achieved by having a Timer Interrupt Service Routine (ISR) increment a tick counter on every timer interrupt, and using the modulus of that counter to raise flags for tasks that are due to run. The main loop (or scheduler) polls these flags and executes the corresponding task, then clears the flag. This program simulates that model: a timer interrupt is emulated with a 100 ms period (usleep(100000)), and three tasks are scheduled at 500 ms, 1 second, and 2 second intervals respectively.

2.2 Source Code

```
#include <stdio.h>
#include <unistd.h>
#include <stdint.h>

volatile uint32_t tick = 0;

volatile int task1_flag = 0;
volatile int task2_flag = 0;
volatile int task3_flag = 0;

// Timer Interrupt Service Routine
void timer_ISR()
{
    tick++;

    // Task 1: every 500 ms
    if (tick % 5 == 0)
        task1_flag = 1;

    // Task 2: every 1 second
    if (tick % 10 == 0)
        task2_flag = 1;

    // Task 3: every 2 seconds
    if (tick % 20 == 0)
        task3_flag = 1;
}

// Task 1
void task1()
{
    printf("Task 1: LED toggled\n");
}

// Task 2
void task2()
{

```

```

    printf("Task 2: Sensor checked\n");
}

// Task 3
void task3()
{
    printf("Task 3: System status checked\n");
}

int main()
{
    printf("=== REAL-TIME TASK SCHEDULER ===\n\n");

    for (int i = 0; i < 40; i++)
    {
        // Generate timer interrupt
        timer_ISR();

        // Execute Task 1
        if (task1_flag)
        {
            task1_flag = 0;
            task1();
        }

        // Execute Task 2
        if (task2_flag)
        {
            task2_flag = 0;
            task2();
        }

        // Execute Task 3
        if (task3_flag)
        {
            task3_flag = 0;
            task3();
        }

        // Timer period = 100 ms
        usleep(100000);
    }

    printf("\nScheduler completed successfully.\n");
    return 0;
}

```

2.3 Program Output

The program was compiled and run on OnlineGDB. The complete console output for all 40 simulated timer ticks is shown below, exactly as produced by the run.

```
=== REAL-TIME TASK SCHEDULER ===
```

```
Task 1: LED toggled
Task 1: LED toggled
Task 2: Sensor checked
Task 1: LED toggled
Task 1: LED toggled
Task 2: Sensor checked
Task 3: System status checked
Task 1: LED toggled
Task 1: LED toggled
Task 2: Sensor checked
Task 1: LED toggled
Task 1: LED toggled
Task 2: Sensor checked
Task 3: System status checked

Scheduler completed successfully.

...Program finished with exit code 0
```

2.4 Scheduling Logic Explanation

- Tick counter: tick is declared volatile because it is modified inside an interrupt-like routine (timer_ISR) and read inside the main loop — the volatile qualifier prevents the compiler from optimizing away repeated reads of a value that can change asynchronously.
- Timer ISR: Every simulated timer interrupt (every 100 ms) increments tick and checks it against three periods using the modulus operator: $\text{tick} \% 5 == 0$ for Task 1 (500 ms), $\text{tick} \% 10 == 0$ for Task 2 (1 s), and $\text{tick} \% 20 == 0$ for Task 3 (2 s).
- Flag-based deferred execution: The ISR itself does not call printf directly — a real ISR should be kept as short as possible. Instead it only sets task1_flag, task2_flag, or task3_flag. The main loop (acting as the scheduler) checks each flag, and if set, clears it and calls the corresponding task function.
- Independent periodicity: Because each task has its own modulus check against the same tick counter, all three tasks are scheduled independently from a single timer source — this is the essence of a simple cooperative, tick-based real-time scheduler.

2.5 Design Analysis

Time complexity: The ISR performs three constant-time modulus checks per tick, so it is $O(1)$ per interrupt regardless of how long the program has been running. The main loop performs $O(1)$ flag checks per iteration, so the overall scheduler is $O(n)$ for n timer ticks, which is optimal since every tick must be processed exactly once.

Space complexity: $O(1)$ — a fixed set of scalar variables (tick and three flags) is used regardless of the number of ticks elapsed; no queues or per-task memory allocation are required.

Real-time suitability:

6. Minimal ISR workload: Keeping the ISR limited to incrementing a counter and setting flags (rather than performing I/O like printf) keeps interrupt latency low and ensures the system remains responsive to the next interrupt — a core real-time design principle.
7. Deterministic scheduling: Because task periods are derived from a common tick source using fixed moduli, the timing relationship between tasks is exact and predictable (Task 2 always coincides with every 2nd occurrence of Task 1, Task 3 with every 4th occurrence), which avoids drift.
8. Volatile-qualified shared state: Marking tick and the flags as volatile ensures correctness when data is shared between an interrupt context and the main program context, preventing stale cached values.
9. Scalability: Additional tasks can be added by introducing a new flag and a new modulus condition without altering the structure of the ISR or the scheduling loop, making the design easy to extend.

This flag-based, tick-driven approach is a lightweight alternative to a full real-time operating system (RTOS) and is commonly used in simple microcontroller applications where only a handful of periodic tasks need to be coordinated from a single hardware timer.

Conclusion

Both programs demonstrate core embedded systems techniques implemented purely in bitwise/logical C constructs. The GPIO program shows how individual hardware pins are controlled with $O(1)$, non-destructive bitwise operations, which is the standard technique used in real microcontroller register access. The scheduler program shows how a single timer interrupt source can be used to drive multiple independently periodic tasks using a tick counter and deferred flag-based execution, a lightweight and deterministic scheduling pattern well suited to resource-constrained real-time embedded applications.